

BIT4107: Project Deployment & Store Listing Report

Academic Profile Header

Course Code & Title: BIT4107 – Mobile Application Development

Student Name: John Njoroge Gachanja

Admission Number: BIT/2024/74648

Application Title: Vault Student Hub

1. Application Overview & Metadata

* Brief Description: A secure, high-performance mobile information utility designed to consolidate student directory management, identity authentication systems, hardware-based geometric telemetry capture (GPS & Camera emulation), and external API synchronization layers into a unified dark-themed Material Design workspace.

* Application Version Name: 1.0.0

* Application Version Code: 1

2. Steps Followed to Generate the Signed Release Build

To transition the application from a local debug canvas into an optimized production-ready binary distribution pack, the following workflow execution layout was carried out:

* Workspace Optimization: Stripped active print telemetry execution statements and development-stage log signatures from the codebase arrays to reduce final package compilation sizing.

* Environment Synchronization: Cleared background package compilation cache structures and executed package synchronization sequences to explicitly link structural material dependencies.

* Platform Dependency Reset: Resolved background dependency conflicts within the local environment path variables to allow seamless execution of the build compiler pipelines.

* Release Compilation Execution: Initiated the native build automation toolchain terminal layer via production-ready release execution flags:

```
flutter build apk --release
```

* Binary Extraction: Located and extracted the compiled production asset distribution file (app-release.apk) directly from the optimized system output build tree.

3. Draft Google Play Store Listing Mock-up

Before publishing binaries to live production registries like the Google Play Console, a standardized distribution asset listing framework was structured:

| Listing Component | Production Metadata Content |

|---|---|

| App Name | Vault Student Hub |

| Short Description | Secure identity verification, integrated student directories, and live hardware telemetry. |

| Long Description | Vault Student Hub is a comprehensive mobile application utility built to modernize campus operations. Featuring an advanced Material Design interface, the system provides cryptographic user authentication barriers, an integrated directory registry featuring real-time case-insensitive filtering search algorithms, immediate device sensor hooks mapping spatial GPS coordinates alongside biometric camera hardware setups, and a dedicated network profiling dashboard that actively monitors cloud-based REST API server connections for production stability. |

| App Icon Asset | High-definition 512 \times 512 pixel PNG graphic showcasing the secure system seal. |

| Graphic Assets | Standard 1024 \times 500 pixel panoramic feature asset alongside contextual user interface screenshots displaying login validation, filtered directory indexes, and active telemetry logs. |

⚠ 4. Deployment Challenges Encountered & Solutions Applied

Challenge 1: Native Development Toolchain Dependency Corruption

* Context: The build engine abruptly terminated compilation routines, throwing an environment failure state tracking back to a malformed or interrupted binary download block within local system folders (Gradle task assembleRelease failed with exit code 1).

* Proposed Solution: Implemented recursive workspace pruning routines directly through terminal commands to nuke the broken local cache directory. This forced the automation toolchain to automatically re-download a verified, uncorrupted file instance upon the next build cycle, instantly fixing the execution layer.

Challenge 2: IDE Project Workspace Configuration Clashes

- * Context: Following workspace resets, the Integrated Development Environment dropped structural configurations, blocking local code playback functions via an explicit entrypoint exception tracking error.

- * Proposed Solution: Bypassed the unstable graphical layout controls entirely by shifting system compilation calls directly onto external command shells, running the terminal commands with explicit device target arguments (`flutter run -d edge`) to run the application code directly.